

dev tra パッケージ編

ミニゲームを1つに集めるための Unity Package 入門（全8回）

【1-01】ミニゲームを1つに集めよう【1章：パッケージ編】

1. クリア条件：何を作るのか、そのために何を提出するのかが分かればOK！
2. ここからは別巻、Unity Packageの使い方です。
作るのは、**各チームが作ったミニゲームを1つにまとめたゲーム**です。タイトル画面から好きなミニゲームを選んで遊べる、ミニゲーム集の形になります。
3. ここで問題があります。ミニゲームは**チームごとに別々のUnityプロジェクト**で作られています。

チームAのプロジェクト ← 迷路ゲーム
チームBのプロジェクト ← シューティング
チームCのプロジェクト ← リズムゲーム
チームDのプロジェクト ← パズルゲーム

これを最終的に、1つのプロジェクトの中に全部入れる必要があります。

統合プロジェクト
├ 迷路ゲーム
├ シューティング
├ リズムゲーム
└ パズルゲーム

4. 「プロジェクトのフォルダごと渡せばいいのでは？」と思うかもしれませんが、それだと**プロジェクトが4つ並ぶ**だけで、1つのゲームにはなりません。
必要なのは「プロジェクトそのもの」ではなく、「**そのミニゲームを動かすのに必要なファイル一式**」を取り出して渡すことです。
5. そのための仕組みが、Unityに用意されている Unity Package です。

用語：Unity Package（.unitypackage）

Unityのプロジェクトから、**指定したファイルをまとめて1つのファイルに書き出したもの**。受け取った側は、それを自分のプロジェクトに読み込むと、フォルダ構成そのままファイルが追加されます。

6. 作業の流れはこうなります。

- ① 各チーム：自分のミニゲームを .unitypackage に書き出す
↓
- ② 各チーム：ファイルとReadMe.mdを提出する
↓
- ③ 迷とら運営：全部を統合プロジェクトに読み込む



④ 迷とら運営：タイトル画面から各ミニゲームへつなぐ

この別巻では、①～②の提出まで（1-02～1-06）と、③の読み込み（1-07）を扱います。

7. 【最重要】提出するファイルの名前は、全チーム統一します。

```
GameJam_2026_07_Team<チーム名>_<ゲームタイトル>
```

チーム名はA～Dなので、こうなります。

```
GameJam_2026_07_TeamA_MazeRunner.unitypackage
GameJam_2026_07_TeamB_ShootingStar.unitypackage
GameJam_2026_07_TeamC_RhythmCat.unitypackage
GameJam_2026_07_TeamD_PuzzleBox.unitypackage
```

※ <> は書きません。Team の後にそのままチーム名（A～D）をつなげます。

※ 半角英数とアンダースコア（_）だけを使ってください。日本語やスペースが入ると、読み込む側で不具合の原因になります。

8. なぜ名前を統一するのか。迷とら運営の画面には、全チームのファイルが並びます。

○ 統一されている場合

```
GameJam_2026_07_TeamA_MazeRunner.unitypackage
GameJam_2026_07_TeamB_ShootingStar.unitypackage
GameJam_2026_07_TeamC_RhythmCat.unitypackage
GameJam_2026_07_TeamD_PuzzleBox.unitypackage
→ どれが誰のもので、いつの提出物か一目で分かる
```

× バラバラの場合

```
game.unitypackage
最新版.unitypackage
test2.unitypackage
→ 中身を開くまで分からない。同名ファイルで上書き事故も起きる
```

9. 目的（ミニゲームを1つに集める）と、提出するもの（決まった名前の.unitypackage）が分かったら、今回は完成です！
次回は、書き出す前の準備をします。

【1-02】書き出す前にフォルダをまとめよう【1章：パッケージ編】

1. クリア条件：自分のミニゲームのファイルが、1つのフォルダにまとまっていればOK！
2. 書き出しの前に、準備が必要です。ここをやらないと、統合するときに全チームのファイルが混ざります。
3. まず、よくない例から見てみましょう。作りながら進めると、だいたいAssetsフォルダはこうなっています。

```
Assets/  
├ Scenes/  
├ Scripts/  
├ Images/  
├ Prefabs/  
└ Audio/
```

一見きれいですが、このまま書き出すと問題が起きます。
統合プロジェクトに4チーム分を読み込むと、こうなるからです。

```
Assets/  
├ Scenes/      ← A・B・C・D チームのシーンが全部ここに入る  
├ Scripts/     ← 4チーム分のスクリプトが混ざる  
├ Images/      ← 4チーム分の画像が混ざる  
└ ...
```

どれが誰のファイルか分からなくなりますし、**同じ名前のファイルがあると上書きされます。**

`Player.cs` や `Title.unity` のような名前は、まず間違いなくかぶります。

4. そこで、書き出す前に**自分のミニゲーム専用のフォルダを1つ作って、その中に全部入れます。**

```
Assets/  
└ GameJam_2026_07_TeamA_MazeRunner/  
  ├── Scenes/  
  ├── Scripts/  
  ├── Images/  
  ├── Prefabs/  
  └ Audio/
```

こうしておけば、統合プロジェクトに読み込んでも、チームごとにきれいに分かります。

```
Assets/  
├ GameJam_2026_07_TeamA_MazeRunner/  
├ GameJam_2026_07_TeamB_ShootingStar/  
├ GameJam_2026_07_TeamC_RhythmCat/  
└ GameJam_2026_07_TeamD_PuzzleBox/
```

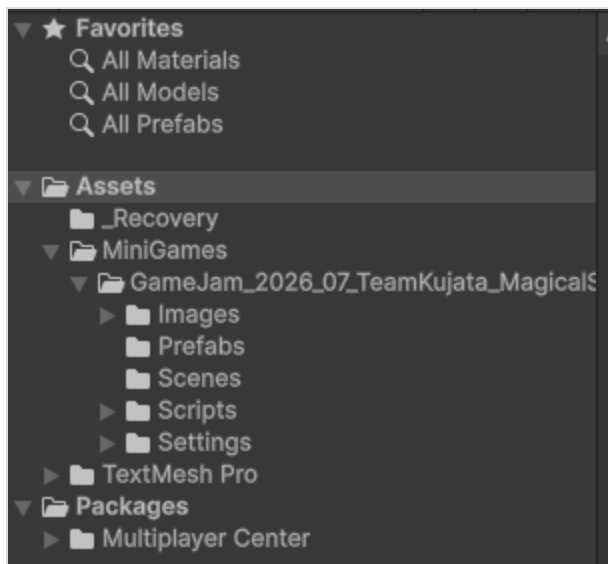
5. フォルダ名は、提出ファイルと同じ名前にしてください。

ファイル名とフォルダ名がそろっていると、迷ったら運営が「このフォルダはどのチームのものか」を確認せずに済みます。

6. では作りましょう。UnityのProjectウィンドウで、Assetsフォルダを右クリック →「Create」→「Folder」。

名前を `GameJam_2026_07_Teamo_ooo` にします。

7. 次に、既存のフォルダをこの中へ移動します。



Projectウィンドウでフォルダをまとめた状態

必ずUnityのProjectウィンドウ上でドラッグして移動してください。

エクスプローラー（Finder）で直接動かすと、Unityが参照を追えなくなって、シーンからスクリプトや画像が外れることがあります。

用語：.metaファイル

Unityが自動で作る、素材とセットの設定ファイル。**Unity上で移動すれば、.metaも一緒に動くので参照は壊れませんが**、エクスプローラーで動かすと.metaだけ取り残されて壊れる場合があります。フォルダの整理はUnity上で、が鉄則です。

8. 移動が終わったら、Unityを再生（Play）して、ゲームがちゃんと動くか確認しましょう。

ここで動かなくなっていたら、参照が外れている可能性があります。**書き出す前に必ず直してください。**

9. 【この時点でよくあるトラブル】

症状	原因と対処
シーンを開いたらオブジェクトが白い四角になった	画像の参照が外れている。Inspectorで画像を割り当て直す
Missing (Mono Script) と表示される	スクリプトの参照が外れている。Inspectorでスクリプトを割り当て直す
Consoleに赤いエラーが出る	エラー内容を読んで対処。まず移動前の状態に戻せないか確認

10. Assetsの中に自分のミニゲーム用フォルダが1つでき、その中に全ファイルが入って、ゲームが動いたら今回は完成です！
次回は、そのフォルダの外に忘れ物がないかを確認します。

【1-03】フォルダの外の忘れ物を確認しよう【1章：パッケージ編】

1. クリア条件：自分のフォルダの外にある「持っていけないもの」に気づけばOK！

2. 前回、ミニゲームのファイルを1つのフォルダにまとめました。

ただし、**Unity Packageに入らないもの**がいくつかあります。ここを知らずに提出すると、統合したときに「自分の環境では動いたのに、統合したら動かない」が起きます。

3. Unity Packageに入るもの／入らないものは、こう分かります。

種類	場所	パッケージに入る？
画像・音・スクリプト・シーン・プレハブ	Assets の中	 入る（選んだものだけ）
Package Manager で入れた機能	Packages	× 入らない
タグ・レイヤー・入力設定・物理設定など	Project Settings	× 入らない
Build Settings のシーソー覧	Project Settings	× 入らない

4. 【確認1】自分のフォルダの外にあるファイルを使っていないか

前回作ったフォルダの外に、使っている素材が残っていないか確認しましょう。

例えば「共通で使うフォントを Assets/Fonts に置いたまま」だと、フォルダごと書き出しても付いてきません。

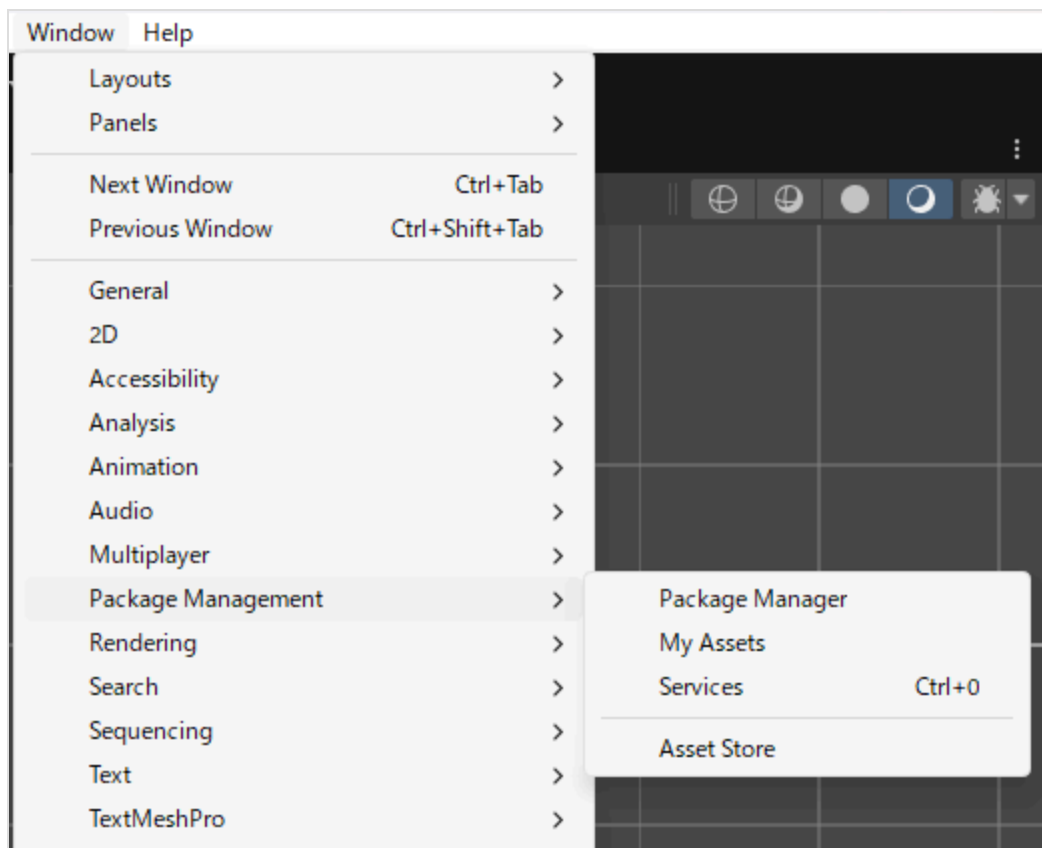
これは書き出しのときにUnityが自動で調べてくれます（次回1-04の「Include dependencies」）。ただし自動でも拾えないものがあるので、下の2つは自分で確認してください。

5. 【確認2】Package Manager で追加した機能

TextMeshPro、Cinemachine、DOTweenなど、後から追加した機能は**パッケージに入りません**。

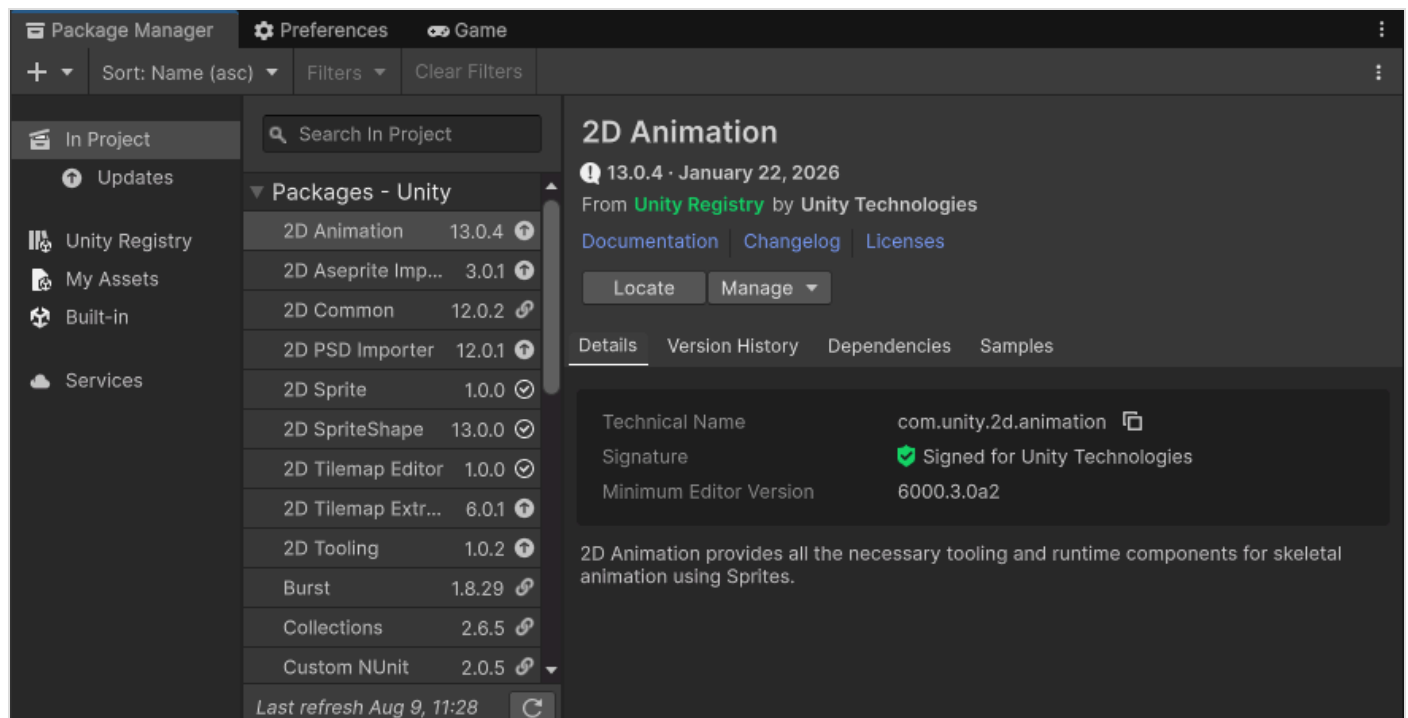
使っている場合は、提出時にReadMe.mdへ書く必要があります（1-06でやります）。

確認するには、**Window > Package Management > Package Manager** を開きます。



Package Managerを開くメニュー

左側の「In Project」を選ぶと、このプロジェクトで使っている機能の一覧が表示されます。



Package ManagerのIn Project表示

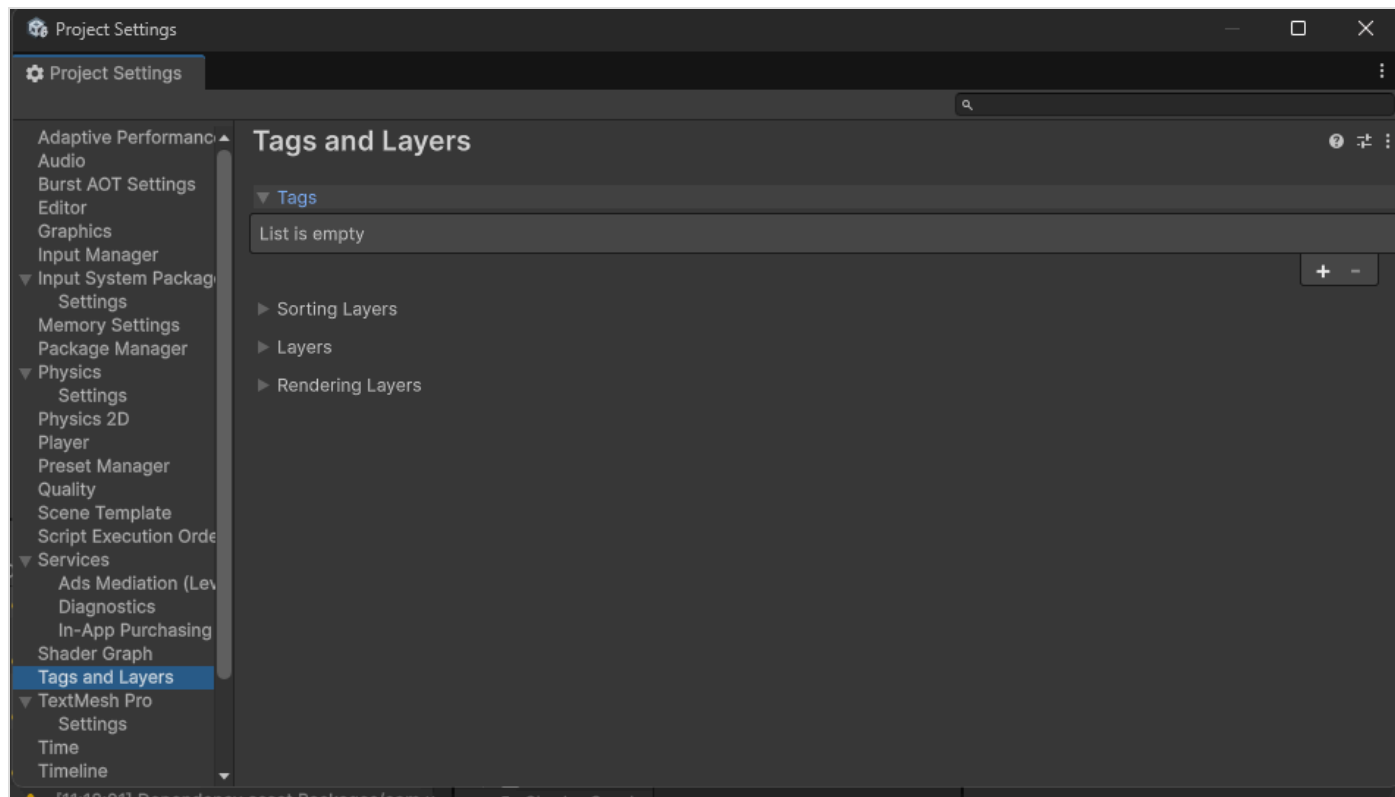
※Unityに最初から入っているものはそのままOKです。自分で追加した覚えのあるものだけメモしておいてください。

6. 【確認3】タグ・レイヤー・入力設定

Project Settings の中身は、パッケージには**一切入りません**。特に次の3つはミニゲームでよく使うので要注意です。

- **Tag（タグ）**：Player や Enemy など、自分で追加したタグ
- **Layer（レイヤー）**：自分で追加したレイヤー、Layer Collision Matrix の設定
- **Input（入力）**：Input Managerで追加した軸の名前、Input Systemの設定ファイル

Edit > Project Settings... を開いて、左のリストから「Tags and Layers」を選びます。「Tags」の欄を開いて、自分で追加したものがあればメモしておきましょう。



Tags and Layersの画面

※スクショのように「List is empty」と表示されていれば、自分で追加したタグは無いということです。この場合は「なし」と書けばOK。

※ Untagged MainCamera Player などの最初からあるタグは、統合先にも最初からあるので大丈夫です。

7. 【対策】スクリプトでタグを使っている場合の注意

例えばスクリプトにこう書いてあるとします。

```
if (other.gameObject.tag == "Item")
```

このとき統合先に Item タグが無いと、実行時にエラーが出てゲームが止まります。

自分で追加したタグは必ず書き残してください。 迷とら運営が統合先に同じタグを追加します。

8. ここでメモした内容は、1-06で作るReadMe.mdにそのまま書きます。

メモしておく項目はこの4つです。

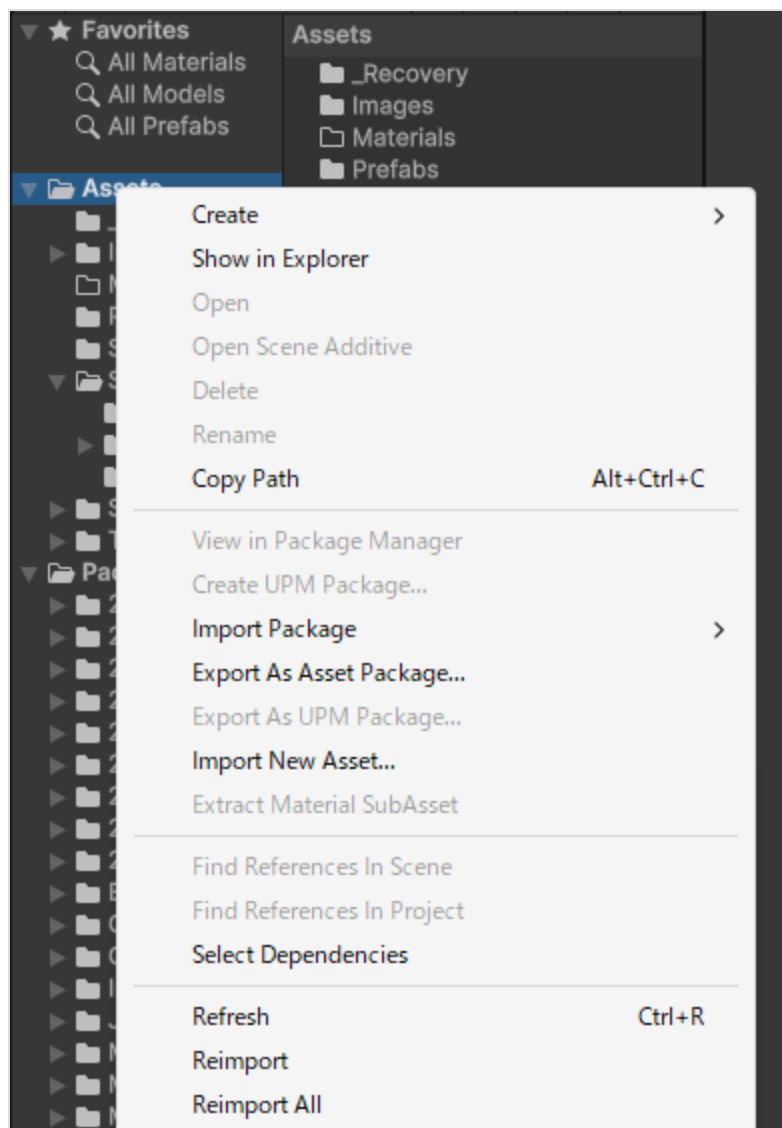
- 追加した機能（Package Managerで入れたもの）
- 追加したタグ
- 追加したレイヤー

- 最初に開くシーンの名前

9. 自分のフォルダの外に何があるか把握できたら、今回は完成です！
次回はいよいよ書き出します。

【1-04】unitypackageに書き出そう【1章：パッケージ編】

1. クリア条件：決められた名前で .unitypackage ファイルが作ればOK！
2. 準備ができたので、書き出します。操作自体は3ステップで終わります。
3. まず、Projectウィンドウで**自分のミニゲーム用フォルダ**を右クリックします。
メニューの中から「Export As Asset Package...」を選んでください。

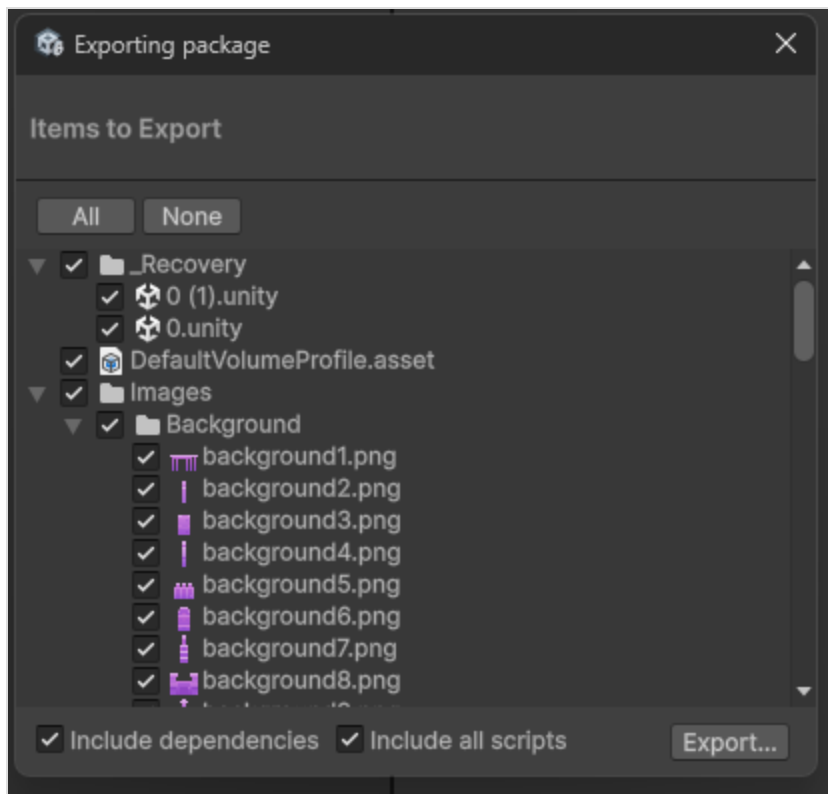


右クリックメニューのExport As Asset Package

※Unityのバージョンによっては「Export Package...」という名前です。どちらも同じ機能です。

※ **Assets** を右クリックすると全部が対象になります。**必ず自分のフォルダを選んでから**右クリックしてください。

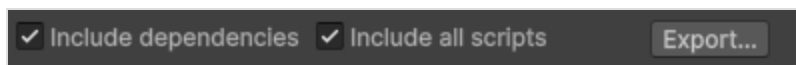
4. 「Exporting package」というウィンドウが開きます。ここが書き出しの設定画面です。



Exporting packageウィンドウ

- 中央のリスト：書き出すファイルの一覧。チェックが付いているものが書き出されます
- **All / None**：全部にチェックを付ける／全部外す
- リストの各行のチェックボックス：個別に外すこともできます

5. 下にある2つのチェックボックスは、**両方チェックを入れたままにしてください**。



Include dependenciesとInclude all scripts

- **Include dependencies**：選んだファイルが使っている他のファイルも自動で追加する
- **Include all scripts**：プロジェクト内のスクリプトをまとめて追加する

用語：Include dependencies（依存関係を含める）

例えばプレハブを1つ選んだとき、そのプレハブが使っている画像やスクリプトも一緒に書き出す設定です。**チェックを外すと、統合先で「絵が付いていない」「スクリプトが無い」状態になります**。特別な理由がなければ、必ずチェックを入れてください。

6. 準備ができたなら、右下の「**Export...**」ボタンを押します。

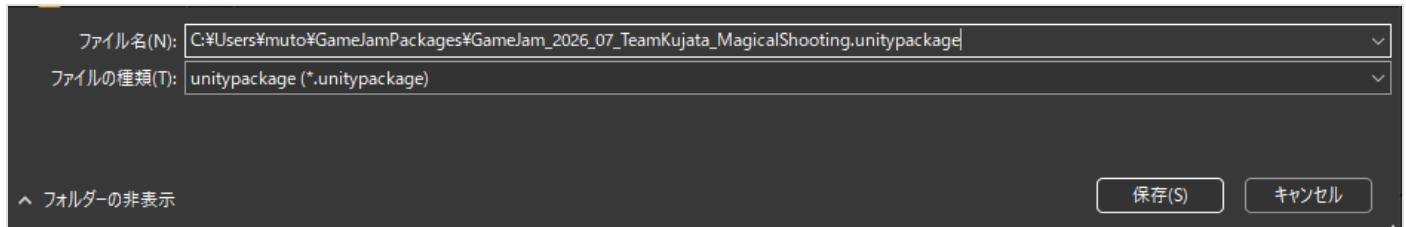
保存先を聞かれるので、分かりやすい場所（デスクトップなど）を選び、ファイル名を入力します。

7. **【最重要】**ファイル名は、決められた形式で付けてください。

GameJam_2026_07_Team<チーム名>_<ゲームタイトル>

例（チーム名はA～D）：

GameJam_2026_07_TeamA_MazeRunner
GameJam_2026_07_TeamB_ShootingStar



保存ダイアログでのファイル名

※スクショは操作画面の例です。実際は自分のチーム名（A～D）とゲームタイトルを入れてください。

※ `.unitypackage` の部分は自動で付くので、入力しなくて大丈夫です。

※半角英数とアンダースコア（`_`）だけを使ってください。

8. 「保存」を押すと書き出しが始まります。プロジェクトの大きさによっては1分ほどかかります。

指定した場所に `.unitypackage` ファイルができていれば成功です。

9. ファイルサイズを確認しておきましょう。

- 数MB～数十MB：正常です
- 数百バイト～数KB：ほとんど中身が入っていません。手順3でフォルダを選び忘れた可能性があります
- 1GB以上：Libraryフォルダなど、本来入らないものが混ざっている可能性があります。書き出す対象を見直してください

10. 決められた名前の `.unitypackage` ファイルができたなら、今回は完成です！

ただし、このまま提出してはいけません。次回、ちゃんと動くかを自分で確認します。

【1-05】提出前に自分で動作確認しよう【1章：パッケージ編】

1. クリア条件：空のプロジェクトに読み込んで、動くか確認できればOK！

2. 書き出したパッケージ、そのまま提出したくなりますが、あと少し待ってください。

自分のプロジェクトでは動くのに、統合先では動かないというのが、パッケージで一番多いトラブルです。

3. 理由は1-03で説明した通りです。パッケージには入らないものがあるからです。

- Package Managerで追加した機能
- タグ、レイヤー、入力設定

自分のプロジェクトにはこれらが全部そろっているのに、当然動きます。でも統合先は別のプロジェクトです。

4. そこで、**空のプロジェクトを作って、自分のパッケージだけを読み込んでみます。**

ここで動けば、統合先でもほぼ確実に動きます。動かなければ、提出前に自分で直せます。

5. UnityHubで「New project」を押して、新しいプロジェクトを作りましょう。

- テンプレート：**自分のミニゲームと同じもの**（2Dで作ったなら Universal 2D）
- Unityバージョン：**チームで決めたバージョン**（6000.3.15f1）
- プロジェクト名：**ImportTest** など、分かるもの

※バージョンが違くと、それだけで動かないことがあります。ここは必ずそろえてください。

6. 新しいプロジェクトが開いたら、**Assets > Import Package > Custom Package...** を選び、書き出したパッケージを読み込みます。

（詳しい手順は1-07で説明します。ここでは読み込んで動かすだけです）

7. 読み込んだら、この4つを確認してください。

確認項目	見るところ
エラーが出ていないか	Consoleに赤いエラーが無い
シーンが開けるか	自分のシーンをダブルクリックして開く
見た目が崩れていないか	白い四角（Missing）になっている絵が無い
実際に遊べるか	Playボタンを押して動かしてみる

8. 【よくあるエラー-1】赤いエラーで「型または名前空間が見つかりません」

実際にやってみたところ、こういうエラーが出ました。

Assets\Scripts\Scene\GameManager.cs(1,13): error CS0234: The type or namespace name 'VisualScripting' does not exist in the namespace 'Unity' (are you missing an assembly reference?)

読み込み後に出たエラー

これは、スクリプトの先頭にある **using** が、統合先に無い機能を指しているときに出了。

```
using Unity.VisualScripting;    // ← この機能が統合先に無い
using Unity.Mathematics;       // ← これも
```

対処は2つあります。

- **その機能を実際に使っていない場合**： `using` の行を消す（多くの場合こちらです。エディタが自動で追加しただけ）
- **実際に使っている場合**： ReadMe.mdに「この機能を使っています」と書く（1-06でやります）

9. 【よくあるエラー2】絵が白い四角になる

.metaファイルと一緒に書き出されていないか、書き出しのときに「Include dependencies」のチェックが外れていた可能性があります。

もう一度1-04の手順で、チェックを入れて書き出し直してください。

10. 【よくあるエラー3】タグ関連のエラーで止まる

`Tag: oo is not defined` のようなエラーが出たら、自分で追加したタグを使っています。

統合先に同じタグを追加してもらう必要があるので、**タグ名をReadMe.mdに必ず書いてください。**

11. 空のプロジェクトで問題なく動いたら、あとは提出用の説明書（ReadMe.md）を書くだけです。

12. 空のプロジェクトで動作確認ができれば、今回は完成です！

次回、提出に必要なReadMe.mdを作ります。

【1-06】ReadMe.mdを書こう【1章：パッケージ編】

1. クリア条件：自分のミニゲームの説明書（ReadMe.md）が作ればOK！
2. パッケージができました。でも、ファイルだけ渡されても迷ったら運営は困ります。
「このゲームはどう操作するのか」「どのシーンから始まるのか」「何を追加したのか」が分からないからです。
3. そこで、説明書のファイルを一緒に提出します。名前は `ReadMe.md` に統一します。

用語：ReadMe（リードミー）

「最初に読んでね」という意味の、説明書ファイルの定番の名前。プログラムの世界では、フォルダの中にReadMeがあれば、まずそれを読むという習慣があります。

用語：.md（マークダウン）

文字だけで見出しや箇条書きを書けるファイル形式。`#` を付ければ見出し、`-` を付ければ箇条書きになります。中身はただの文字なので、メモ帳でも作れます。

4. まずは作り方から。メモ帳（Windows）またはテキストエディット（Mac）で作れます。

【Windowsの場合】

1. メモ帳を開く
2. 後で出てくるテンプレートを貼り付けて、中身を書き換える
3. ファイル > 名前を付けて保存
4. ファイルの種類を「すべてのファイル」に変更
5. 文字コードを「UTF-8」に変更
6. ファイル名を `ReadMe.md` にして保存

※手順4をやらないと `ReadMe.md.txt` という名前になってしまいます。

※手順5をやらないと、日本語が文字化けすることがあります。

【Macの場合】

テキストエディットは初期設定だとリッチテキストになるので、フォーマット > 標準テキストにする をしてから保存してください。

※VSCodeを使っている人は、新規ファイルを作って `ReadMe.md` の名前で保存するだけでOKです。

5. 置き場所は、自分のミニゲームのフォルダの中です。

```
Assets/  
├─ GameJam_2026_07_TeamA_MazeRunner/  
│   ├── ReadMe.md      ← ここに置く  
│   ├── Scenes/  
│   ├── Scripts/  
│   └─ ...
```

ここに置いてからパッケージを書き出せば、ReadMe.mdもパッケージに含まれます。

※すでに書き出し済みの人は、ReadMe.mdを置いてからもう一度1-04の手順で書き出し直してください。

6. では、中身を書きます。下のテンプレートをそのままコピーして使ってください。

```
# ゲームタイトル

## ゲーム概要
（どんなゲームか2〜3行で。ジャンル、目的、勝敗条件など）

## 操作方法
- 移動：
- 決定：
- その他：

## 最初に開くシーン
（例：Title_TeamA.unity）

## 追加した機能（Package Manager）
- （例：TextMeshPro）
- 無ければ「なし」

## 追加したタグ
- （例：Item）
- 無ければ「なし」

## 追加したレイヤー
- （例：Ground）
- 無ければ「なし」

## その他
- 想定プレイ時間：
- 終了条件：（どうなったらゲームが終わるか）
- 使用素材：（配布サイトの名前とURL。自作なら「自作」）
- 既知の不具合：
- 制作メンバー：
- 連絡先：（DiscordのID。統合中に質問が出たとき用）
```

7. 書き方の例です。テンプレートを埋めるとこうなります。

```
# 迷路ダッシュ

## ゲーム概要
制限時間内にゴールを目指す2D迷路ゲームです。
道中のアイテムを取ると制限時間が5秒増えます。
時間切れになるとゲームオーバーです。

## 操作方法
- 移動：矢印キー
- 決定：スペースキー
- その他：Escキーでタイトルに戻る

## 最初に開くシーン
```

```
Title_TeamA.unity

## 追加した機能 (Package Manager)
- TextMeshPro
- Cinemachine

## 追加したタグ
- Item
- Goal

## 追加したレイヤー
- なし

## その他
- 想定プレイ時間：2分
- 終了条件：ゴールに触れるとクリア、制限時間0でゲームオーバー。どちらもタイトルに戻ります
- 使用素材：効果音ラボ (https://soundeffect-lab.info/)、キャラ絵は自作
- 既知の不具合：まれに壁をすり抜けることがあります
- 制作メンバー：〇〇、△△
- 連絡先：Discord @maruo
```

8. 各項目について、迷ったら運営が何に使うかを説明します。ここが分かると書きやすくなります。

項目	運営がこう使う
ゲームタイトル	タイトル画面のメニューに表示する名前になる
ゲーム概要	遊び方が分からないときに読む。紹介文にも使う
操作方法	動作確認するときに必要。遊ぶ人への案内にも使う
最初に開くシーン	タイトル画面からこのシーンへつなぐ。これが無いと組み込めない
追加した機能	統合プロジェクトに同じ機能を入れる。無いとエラーで動かない
追加したタグ	統合プロジェクトに同じタグを追加する。無いと実行時に止まる
追加したレイヤー	同上。当たり判定が絡むので、抜けると挙動が変わる
終了条件	ミニゲームが終わったらタイトルに戻す処理を入れるときに必要
使用素材	配布サイトによっては表記が必要。まとめてクレジットに載せる
既知の不具合	統合後に発覚したとき「元からか、統合で壊れたか」の判断材料になる
連絡先	統合中に不明点が出たとき、すぐ聞ける

9. 【書くときの注意】

- 「なし」も書く。空欄だと「書き忘れ」なのか「無い」のか分かりません
- 追加したタグ・レイヤーは、1-03で確認した内容をそのまま書けばOK
- シーン名は拡張子（.unity）まで正確に。Title と title の違いでも動きません

10. 書けたら、提出です。パッケージファイルとReadMe.mdの両方を提出してください。

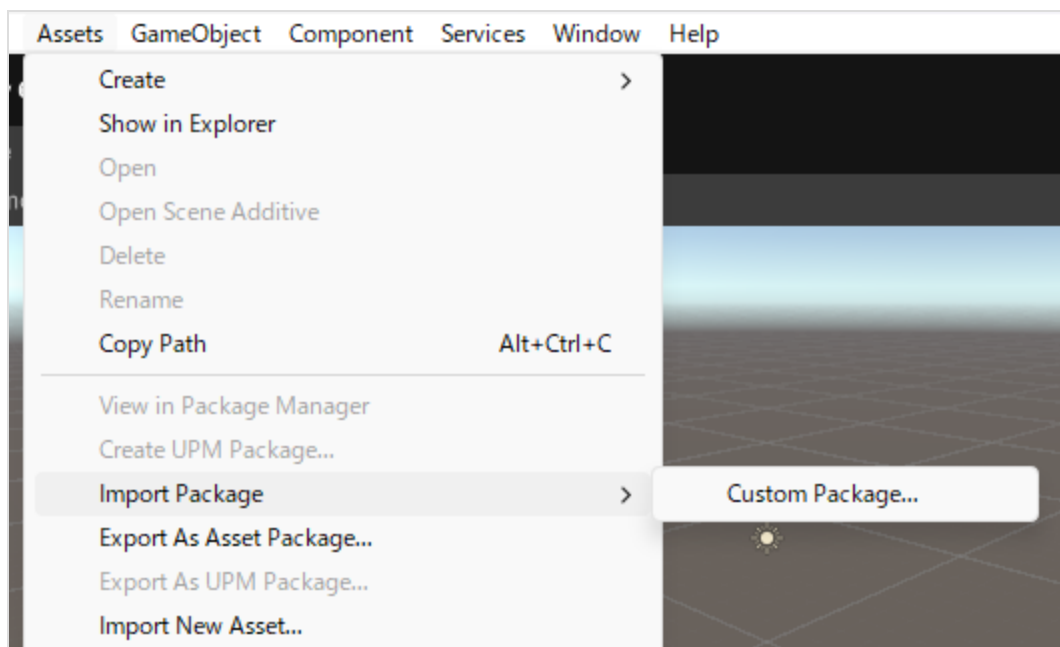
ReadMe.mdの中身は、Discordにそのまま貼り付けてもらえると運営が確認しやすいです。

11. ReadMe.mdが作れて、提出できたら、今回は完成です！

ここまでで、書き出す側の作業は終わりです。次回は受け取る側（迷とら運営）の作業を見ていきます。

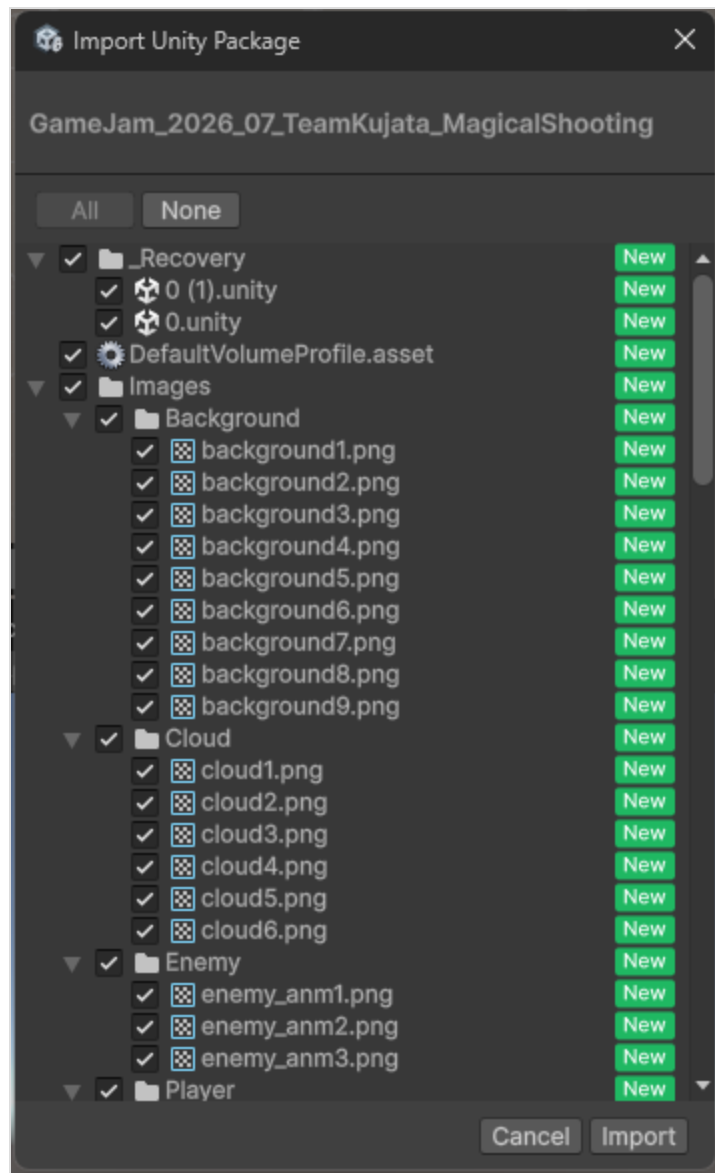
【1-07】統合プロジェクトに読み込もう【1章：パッケージ編】

1. クリア条件：受け取ったパッケージを統合プロジェクトに読み込めればOK！
2. ここからは受け取る側、迷とら運営の作業です。
※自分が運営でない人も、目を通しておいってください。自分の提出物がどう扱われるかが分かったら、提出物の作り方が変わります。
3. 全チームのパッケージがそろったら、統合プロジェクトを開きます。
まだ無ければ、UnityHubで新規プロジェクトを作ってください（テンプレートとバージョンは、チームで決めたものにそろえます）。
4. Assets > Import Package > Custom Package... を選びます。



Import Packageメニュー

5. ファイル選択の画面が出るので、1チーム分のパッケージを選びます。
※必ず1つずつ読み込んでください。まとめて読み込むと、問題が起きたときにどのチームのものが原因か分からなくなります。
6. 「Import Unity Package」というウィンドウが開きます。中身の一覧が表示されるので、そのまま右下の「Import」を押します。

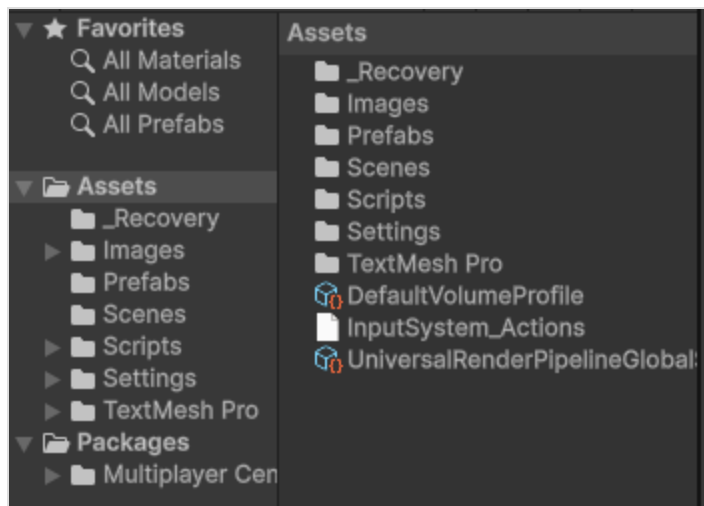


Import Unity Packageウィンドウ

- 上部にパッケージ名が表示されます。ここで名前を確認しておくと、どのチームのものか分かります
- 緑の「New」は新しく追加されるファイル、「Changed」は既にあるファイルが上書きされるという意味です

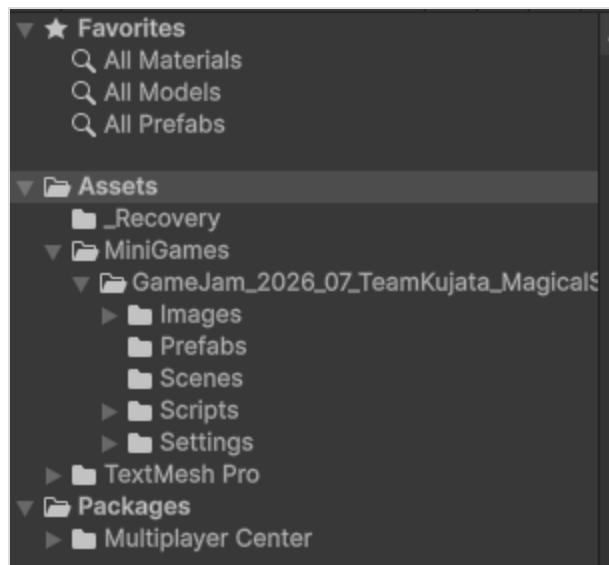
※「Changed」が出たら要注意です。他のチームのファイルを上書きしようとしている可能性があります。その場合はImportを押さず、提出したチームに確認してください。

7. 読み込みが終わると、Projectウィンドウにファイルが追加されます。



読み込み後のProjectウィンドウ

1-02でフォルダをまとめてもらっているので、チームごとにきれいに分かれているはずです。



チームごとに分かれた状態

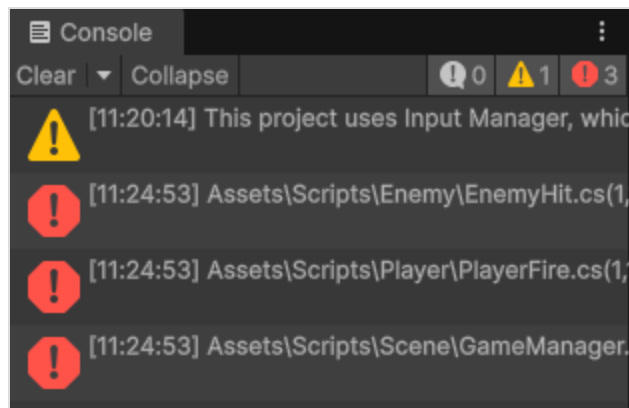
8. 読み込んだら、まず**そのチームのReadMe.md**を開いてください。

- 追加した機能 → 統合プロジェクトにも同じものを入れる
- 追加したタグ・レイヤー → `Edit > Project Settings > Tags and Layers` で追加する
- 最初に関くシーン → タイトル画面からつなぐ先として控えておく

ここを先にやっておくと、次のエラー確認がスムーズになります。

9. 【ここからが本番】Consoleを確認する

赤いエラーが出ていたら、そのチームのミニゲームはまだ動きません。



読み込み後に出たエラー

実際にやってみたところ、3つのエラーが出ました。内容はこれです。

```
Assets\Scripts\Scene\GameManager.cs(1,13): error CS0234: The type or namespace name 'VisualScripting' does not exist in the namespace 'Unity' (are you missing an assembly reference?)
```

エラーの詳細

```
error CS0234: The type or namespace name 'VisualScripting'
does not exist in the namespace 'Unity'
```

これは「スクリプトが使っている機能が、このプロジェクトに入っていない」という意味です。1-03で説明した「パッケージに入らないもの」が原因です。

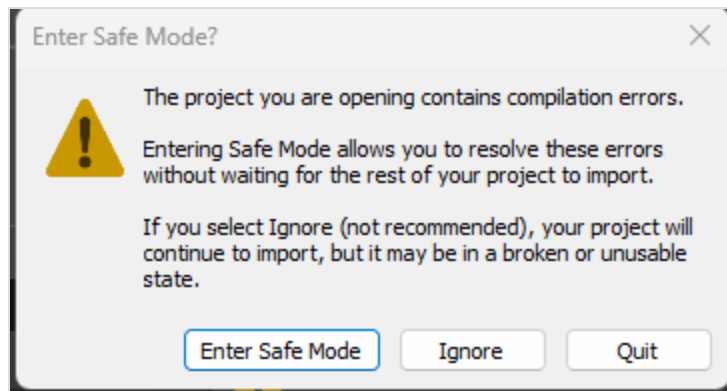
10. 【対処】足りない機能を追加する

Window > Package Management > Package Manager を開いて、エラーに出ている機能を追加します。

または、その機能を実際には使っていない場合、スクリプトの `using` の行を消せば直ります（提出したチームに確認してから消しましょう）。

11. 【注意】エラーがあるまま再起動すると、Safe Modeになる

コンパイルエラーが残った状態でUnityを開き直すと、この画面が出ます。



Enter Safe Mode?ダイアログ

- **Enter Safe Mode**：エラーを直すことに集中できるモードで開く
- **Ignore**：とりあえず無視して開く（エラーは残ったまま）

どちらを選んでも、エラーを直さない限りゲームは動きません。エラーは読み込んだその日に直すのが鉄則です。

12. 【全チーム分を読み込む手順】

1チーム読み込む → ReadMe.mdの内容を反映 → エラーを確認 → 直す → 動作確認 → **次のチームを読み込む**
この順番を必ず守ってください。まとめて読み込むと、エラーが混ざって原因が分からなくなります。

13. 【最後に】シーンをBuild Settingsに登録する

各ミニゲームのシーンは、読み込んだだけではビルドに含まれません。

File > Build Profiles を開いて、Scene Listに全ミニゲームのシーンを追加してください。

※ReadMe.mdの「最初に開くシーン」が、ここで役に立ちます。

14. 全チームのパッケージを読み込めて、エラーが無くなったら、今回は完成です！

次回は最終回、提出前のチェックリストをまとめます。

【1-08】通し：提出チェックリスト【保存版】

1. クリア条件：これ1枚を見れば、提出まで迷わず進められればOK！
2. 最終回です。ここまでの内容を忘れても、このページさえ開けば提出できるようにまとめました。
Discordにピン留めするなどして、手元に置いてください。

【A】提出までの流れ

- ① Unityで自分のミニゲーム用フォルダを1つ作る
名前：GameJam_2026_07_Team<チーム名>_<ゲームタイトル>
↓
- ② そのフォルダの中に、自分の素材・スクリプト・シーンを全部移動
(移動はUnityのProjectウィンドウ上で！)
↓
- ③ Unityで再生して、まだ動くか確認
↓
- ④ 追加した機能・タグ・レイヤーを確認してメモ (1-03)
↓
- ⑤ ReadMe.md を書いて、自分のフォルダの中に置く (1-06)
↓
- ⑥ フォルダを右クリック → 「Export As Asset Package...」
↓
- ⑦ Include dependencies にチェックが入っているか確認 → Export
↓
- ⑧ ファイル名を決められた形式で保存
↓
- ⑨ 空のプロジェクトを作って読み込み、動作確認 (1-05)
↓
- ⑩ パッケージ+ReadMe.mdを提出

【B】ファイル名とフォルダ名のルール

GameJam_2026_07_Team<チーム名>_<ゲームタイトル>

良い例 ○	避けたい例 ×
GameJam_2026_07_TeamA_MazeRunner	game.unzippackage
GameJam_2026_07_TeamB_ShootingStar	最新版.unzippackage
GameJam_2026_07_TeamC_RhythmCat	GameJam 2026 07 チームC.unzippackage
GameJam_2026_07_TeamD_PuzzleBox	teamD_final_ver2.unzippackage

- <> は書かない (Team の後にそのままチーム名A～D)

- 半角英数とアンダースコア (_) だけを使う
- 日本語・スペース・記号は使わない
- フォルダ名もファイル名と同じにする

【C】パッケージに入るもの・入らないもの

種類	入る？	対応
画像・音・スクリプト・シーン・プレハブ	○	自分のフォルダにまとめる
Package Managerで追加した機能	×	ReadMe.mdに書く
タグ・レイヤー	×	ReadMe.mdに書く
入力設定（Input）	×	ReadMe.mdに書く
Build Settingsのシーソー覧	×	「最初に開くシーン」をReadMe.mdに書く

【D】ReadMe.md テンプレート（そのままコピーして使えます）

```
# ゲームタイトル

## ゲーム概要
（どんなゲームか2〜3行で。ジャンル、目的、勝敗条件など）

## 操作方法
- 移動：
- 決定：
- その他：

## 最初に開くシーン
（例：Title_TeamA.unity）

## 追加した機能（Package Manager）
- 無ければ「なし」

## 追加したタグ
- 無ければ「なし」

## 追加したレイヤー
- 無ければ「なし」

## その他
- 想定プレイ時間：
- 終了条件：（どうなったらゲームが終わるか）
- 使用素材：（配布サイトの名前とURL。自作なら「自作」）
- 既知の不具合：
```

- 制作メンバー：
- 連絡先：（DiscordのID）

- 保存するときは、ファイル名 `ReadMe.md`、文字コード **UTF-8**
- 置き場所は**自分のミニゲームのフォルダの中**（パッケージと一緒に入ります）
- 「なし」も必ず書く（空欄だと書き忘れと区別が付きません）

【E】提出前チェックリスト

- ☐ 自分のミニゲームのファイルが、1つのフォルダにまとまっている
- ☐ フォルダ名が決められた形式になっている
- ☐ フォルダを移動したあと、Unityで再生して動いた
- ☐ ReadMe.md を書いて、フォルダの中に置いた
- ☐ 「Include dependencies」にチェックを入れて書き出した
- ☐ ファイル名が決められた形式になっている
- ☐ ファイルサイズが極端に小さくない（数KBは中身が空の可能性）
- ☐ 空のプロジェクトに読み込んで、エラー無しで動いた
- ☐ パッケージとReadMe.mdの両方を提出した

【F】困ったとき逆引き表

こうなった	こうする
書き出したファイルが数KBしかない	フォルダを選ばずに書き出した可能性。自分のフォルダを右クリックしてやり直す
読み込んだら絵が白い四角になる	Include dependencies のチェック漏れ。チェックを入れて書き出し直す
型または名前空間が見つかりません エラー	統合先に無い機能を使っている。使っていないなら該当の <code>using</code> を消す。使っているなら ReadMe.mdに書く
Tag: <code>oo is not defined</code> エラー	自分で追加したタグを使っている。ReadMe.mdに書いて、統合先にも追加してもらう
Unityを開いたら Enter Safe Mode? が 出た	コンパイルエラーが残っている。Ignoreで開いて、Consoleの赤いエラーを直す
統合先で他チームのファイルを上書きしそ う	Importウィンドウに「Changed」が出ている。Importせず提出チームに確認
フォルダを移動したら参照が外れた	エクスプローラーで移動した可能性。Unity上で移動し直す
統合先でシーンが起動しない	Build ProfilesのScene Listにシーンが追加されていない
ReadMe.md が ReadMe.md.txt にな った	メモ帳の保存時に「ファイルの種類：すべてのファイル」を選び忘れ。名前を変更する
ReadMe.md の日本語が文字化けする	保存時の文字コードがUTF-8になっていない。UTF-8で保存し直す

1. お疲れさまでした！これでパッケージ編は完了です。

ポイントは4つだけです。

- 1つのフォルダにまとめてから書き出す
- 決められた名前を付ける
- ReadMe.mdを書く
- 空のプロジェクトで動作確認してから提出する

この4つを守れば、統合作業はとてもスムーズになります。良いゲームジャムを！